

New Software Development Paradigms and Possible Adoption for Security

Matthias Günter
Swiss Federal Institute of
Intellectual Property
matthias.guenter@ipi.ch

Michael Gisler
Swiss Federal Office of
Information Technology,
Systems and
Telecommunication
michael.gisler@bit.admin.ch

Bianca von Bredow
Dept. of Computer Science,
University of Zurich
bvonbred@ifi.unizh.ch

Abstract

EGovernment would be a lot easier if it could be handled like eBusiness. Unfortunately there are organizational and legal obstacles that make it virtually impossible. The security level needed for public administrations is usually at the level of top private sector industries and service organizations like banks. In project management there has been a paradigm shift towards new methodologies for the development of software. This article studies some possibilities along the same lines with regard to security. We will use extreme programming (XP) as an example of the new software development paradigms.

1. Introduction¹

”Adding manpower to a late project makes it even later”, as the saying goes for projects. The same holds true for security when the responsible team does not do its job properly. The situation gets even worse, because the real security level does not keep up with the security feeling. In this paper, we define security as follows:

Computer security is the area concerned with all questions of the security of IT systems. Especially attacks must be prevented and physical events (e.g. like fire, water, earth quakes) must be handled and the biggest obstacles possible are to be put into the attackers’ way.[1]

Therefore security has a lot to do with risk analysis, prevention and impact reduction. The definition on eGovernment is the following:

EGovernment embraces in its function as regulating eGovernment the creation of the framework for an information society and, as participating eGovernment, the application of new IT and communication technologies by the public sector. [2]

This paper will illustrate several critical issues in eGovernment and address approaches to handle them. For

the purposes of this paper it is not necessary to distinguish G2G (government-to-government) and G2C (government-to-citizen). Both front-office and back-office have software development ongoing. The specific security issues will vary, but can be addressed the same way. However, it focuses on the participating eGovernment part of the definition above. Many current projects - whether in eGovernment or eBusiness - have faced the following situation:

- The governmental body has already been using IT for a long time.
- There is an installed base of hosts around which the architecture was designed .
- The host security is rather high, but it was based on the assumption that the hosts would be isolated from public networks.
- Due to the fact that the governmental body has opened up client connections to the Internet, the internal network has grown enormously.
- The governmental body has a conservative technology strategy.
- The governmental body has installed a fire wall.
- So far the governmental body has been trying to cope with the regulations set out by the responsible administrative branches (dealing with security and other) of the government, while trying to reposition itself in new public management and to increase competitiveness. The security regulations are mainly regarded as being too tight, having nothing to do with reality , but instead they reduce the effectiveness for service provision.
- A consultant is hired to analyse the situation, and he presents a paper about the necessity to create a task force in order to draft a policy or a set of policies.
- A lot of stakeholders are ignored in the process, because there is no time or the stakeholders are thought of as being incapable of providing useful input. It is important to note that in public administrations a lot more people are involved in most activities. Therefore the range of stakeholders is wider than in the private sector.

¹ This paper reflects the personal opinions of the authors.

- The result: little has been done, and because of this many people have become angry, and the widest security gaps remain open (usually internal organisation problems are of this category that is underestimated the most).

Often security is an issue that is unwillingly discussed (meaning with a too low priority), handled the wrong way or the solution possibilities are not followed to the end. A few more concepts must be briefly discussed in this paper. We will refer to security functions in different sections in this article.

Security function — A part or parts of the TOE (target of evaluation) that have to be relied upon for enforcing a closely related subset of the rules originating from the TSP (TOE Security Policy). [3]

Another important concept is that of stakeholders:

Stakeholders are those who have an interest in a particular decision, either as individuals or representatives of a group). This includes people who influence a decision, or can influence it, as well as those affected by it. [4]

The next important one is the definition of a system:

A system is a collection of elements that are in relation to each other. A system is a system, because it consists of connected elements, that in certain ways form a whole. [5]

An application is an information system (IS). This means that several applications may share the same information technology system (IT).

For low to medium level security there are directly implementable solutions. In Switzerland there is the directive S.02 [6] and in Germany the Grundschriftzhandbuch [7].

For more advanced and complete solutions the following two standards should be considered:

- The Common Criteria for Information Technology Security Evaluation
- And ISO 17799 Information technology — Code of practice for information security management.

These two standards are made for specialists and should only be applied under the right circumstances. They can be adapted to fit the method suggested in the following sections as starting point, guidance and check list.

2. What is special in eGovernment security? [8]

eGovernment is special because of the following facts:

- It is governed by different laws as compared to eBusiness.

- The big data pools of the administrations are an attractive goal for an attack and need the appropriate protection.
- The huge and distributed networks of the public administration offer a big attack region.
- Federalism and autonomy of different branches result in a heterogeneous and complex environment.
- It will be difficult enough to offer competitive (instead of merely paper) solutions without enforcing rigid security for citizens.
- The transactions and the processes of the administration are more closely monitored and must adhere to more regulations.

It should be made clear that one of the goals of eGovernment must be to diminish the differences between eGovernment and eBusiness so that the total cost of ownership of eGovernment solutions decreases.

The security issues can affect citizens (because it is often data about them) and on the other hand the agencies. The inability to pay or to do business has not the same importance as in normal business. However, the long-term storage of data can be affected. On a higher scale of escalation the possibility to react appropriately to external dangers (terrorism, war) must be maintained by the relevant parts of the government.

3. Can security be built in from the start?

Security can certainly be "built" into the minds of an organisation. The difficulties to change an inadequate culture [9] in a company are, however, very high and need a long time to be sorted out [10]. It is easier to work on technology, but it is impossible to overcome a glass ceiling level of security when the organisational aspects are not taken into account [11]. In some cases the problem of having to add the security issue last in a particular organisation (together with quality and performance) cannot be avoided as many applications are bought from the shelf and are not of the required standards to cope with the risks present in the environment of public administration. In a high security governmental environment this is even true for commercial operating systems [12,13]. Unfortunately the security architecture of most applications is neither modular nor does it offer a public, stable interface. Attempts in this direction are made, but still not satisfactory [14,15]. This may be one of the side-effects of the export restrictions of cryptographic software [16].

4. Integration of predefined security into an overall scheme

Most of the current systems have a security policy of their own. In a low to medium security environment it is

best to integrate the use of these building blocks. When high level security is needed, there will certainly be problems:

- The security framework of the applications/systems was too narrowly defined.
- There are no usable interfaces to integrate the security architectures.

Both shortcomings are not really the result of bad design, but a design focused on something else besides the security of governmental bodies. A vendor can hardly be blamed for not having foreseen these problems.

5. Security is an aspect of management

It has already been mentioned before that security is a 'people problem'. In each organisation the part dealing with people is the management. Therefore security is a management problem, more precisely, it is an *aspect* of management. It is a bad idea to separate aspects into different categories, security officers and quality officers have a low impact because the power is still and will still be exercised by the default hierarchy. It is absolutely necessary that each manager is instructed about the importance of security and has incorporated the security aspect into his management style. Also, here the gap between middle management and top management is important to keep in mind. Usually the middle management is most opposed to new security strategies as it must implement it and security is often (or is often seen) in conflict with the productivity goals.

The most important security issues for the management to handle are:

- Alignment with other goals
- Homogeneity of usage
- Long lasting measures
- Feedback circles

These are primarily soft factors, but in the end soft factors have a much greater impact.

6. Process-oriented view

One or more anchors are needed to build security into an organisation so that security becomes a core value of these organisations. One organisational view which has become very popular is a process or a workflow-based view. When the organisation is constructed by "organisation follows process follows strategy", a process view on security comes in natural. The basic definition is:

A business process is a collection of interrelated work tasks, initiated in response to an event that achieves a specific result for the customer of the process. [17]

On the other hand: A workflow is characterised by the following attributes [18]:

- Several persons are involved in the execution of a work flow.
- A work flow consists of several tasks.
- The tasks of a work flow have a predefined structure to process matters related to an organisation.
- The handling of errors is predefined.
- There is an instance that coordinates the different tasks.
- During the execution of a work flow data is transferred and managed.

Business processes can be seen as special cases of a workflow. Another view is to follow the life cycle of the applications as shown in figure 1 [19]. An example for an architecture centred security solution is shown in figure 2 [20]. Balancing security and usability is one of the crucial points [21]. Especially when the security is added late in the life cycle of products [22], it will almost certainly affect productivity. Generally it is assumed that a graph as represented in figure 3 (a) is able to describe the connection. What is not realised is that at a certain level of annoyance the users will start to circumvent the security.

Therefore the graph should be more like the presentation in figure 3 (b).

Hard facts on this correlation cannot be found [23]. On the other hand a lot can be done without affecting the users or even doing the opposite, e.g. single-sign on, intrusion detection systems, re-organising the network, a better password policy, physical access restrictions to servers, a good network topology.

7. The top-down and strategy first approach

This is the method most literature proposes. However, this approach has the same drawbacks as the old waterfall model [24]. A lot of paper is produced which makes a good impression on the top management. Usually several months (to years) are lost, while the biggest gaps remain still open. The produced paper is in most cases neither people-focused nor technology-focused. Usually from the point of view of strategy, a fast jump to conventional good practice is made. This also happens often in eGovernment in project management as shown by Teresa A. Pardo and Jochen H. Scholl [25]. Security policy documents incorporating this shortcut can be recognised by the fact that the level of abstraction is incoherent, i.e. that the length of passwords is mentioned on the same page as the need for security training for the staff. A first jump does not need to be bad with regard to security, but it affects the confidence in the process to some extent and for others creates a negative feeling to have followed a systematic approach (which they in fact have not).

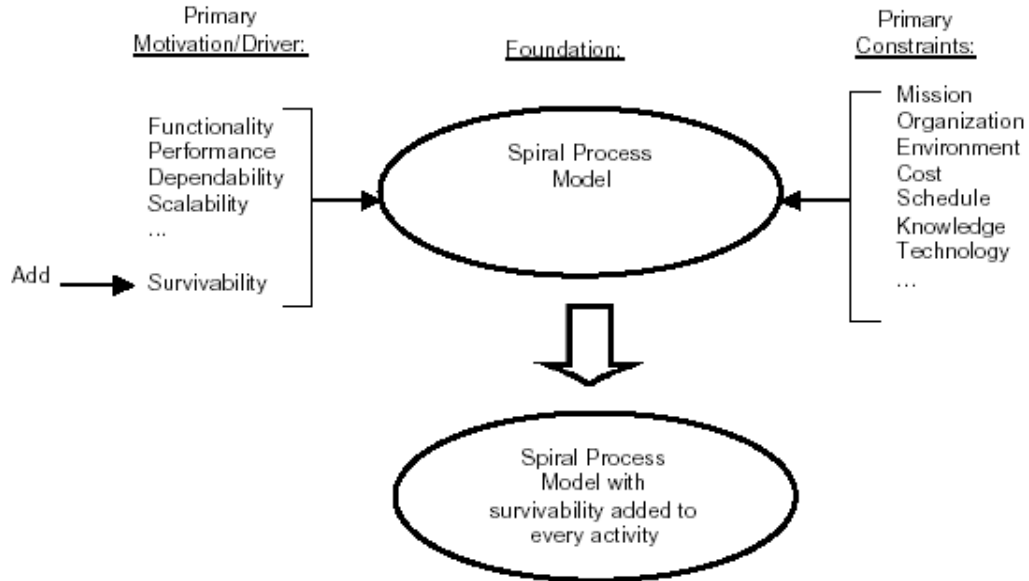


Figure 1: Security and the life cycle of applications

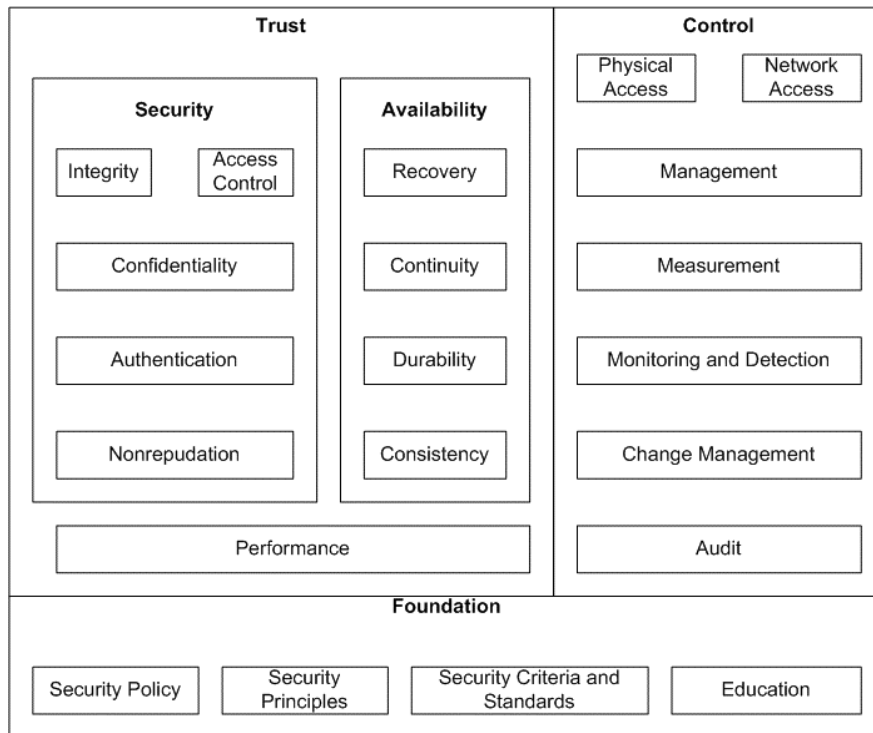


Figure 2: Security architecture model

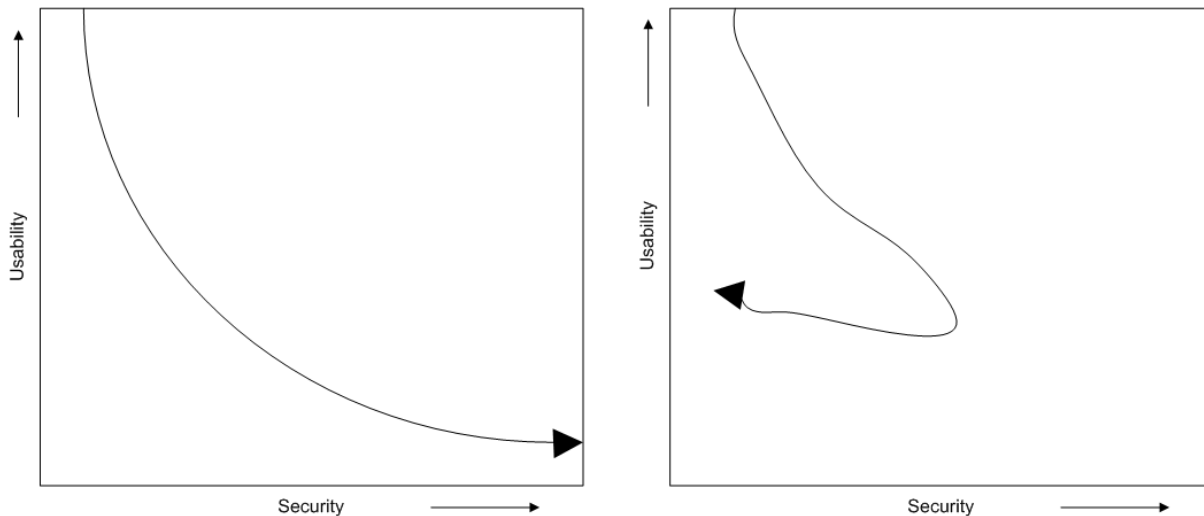


Figure 3: (a) Assumed relation between usability and security (b) Probable relation between usability and security.

8. Integration into a bigger framework

Inter-organisational trust always causes problems, as the non-existence of a world wide certification authority framework shows. Many organisations seem to believe in the value of medieval castles, because in that very same way, they organise their defence for the information infrastructure. More advanced companies already use the fortress engineering of the 18th century by building whole fortifications. State of the art systems should not trust their peers (trust is good, control is better). While some trust in the form of loyalty can be assumed within an organisation that has at least a physical location, a culture and one management; nevertheless a small amount of paranoia between organisations can be healthy. All data traversing the borders should be controlled again and the outside world should be mistrusted. In general this creates an extra amount of work, but it helps at the same time to reduce trouble with other problems like corrupt data and manipulation errors. It is strongly advised that in the case of an economically unbalanced situation the stronger partner does not force the smaller ones to conform, because in many cases the attack will come through the network of the bigger one (with more staff being responsible for 80% of the attacks) and the enforced security will not be incorporated. Security must be a group effort as was mentioned above. It needs only one grumpy stakeholder (person or organisation) to be weakened completely.

9. Extreme programming core practices and security

In this section we will discuss the connections with one well known exponent of the new paradigms: extreme programming is one of the various iterative methods for the development of software [26].

Extreme programming (XP) is a compact method for the development of software in small to medium size teams whose work is dominated by vague or fast changing requirements. [27]

The advantages of modern forms of software development have been shown in business. They are still quite uncommon in the governmental area. The paper wants to show, that - in the area of problems suitable for XP - XP can be used to create adequate software with adequate security levels. The XP process and metaphors can even be used/adapted for fast implementation of security (either technical or organisational) in an already installed environment. The rationale behind such a process is to incorporate security in the core development process and to get the benefits that XP and other adaptive/iterative strategies brought to software development also for the security area.

Extreme programming uses four variables: time, cost, quality, functionality. For Extreme programming time is a constant. Security consists mainly of quality and functionality. Therefore it is a reasonable assumption to examine XP principles for security projects. In table 1 we

will discuss what happens if XP core practices are applied to security. Core practices are behaviour patterns that must be implemented to get XP going.

Table 1 uses the following notations: CO (comment), SE (security effects of applied XP), IM (used an XP analogue for security implementation), EGOV (special eGovernment issue).

Core practice and rules	Security adapted version
Whole team: all the contributors to an XP project sit together as members of one team. The team must include a customer.	CO: This is in some organisations hard to implement. However, it increases the productivity and gets projects working properly. SE: Better collaboration and better understanding will make the security training much more easy. IM: Security created by a team that has all the skills and involves the users and customers with each other will more likely succeed in the roll-out. EGOV: Public administration environments are a bit more defensive towards projects and teams.
Planning Game: The planning game defines the next iteration's goals in an adaptive manner with stories. Also the releases are planned that way. The process is rather straightforward. A full description can be found in the books of Kent Beck. [28]	CO: The planning game is one of the more important concepts of extreme programming. It is very powerful in ensuring the dialogue. SE: The planning game creates the possibility to add security features in all the stories and give them the necessary priority. IM: The planning game is also powerful to prioritise the work of a security team. The top priorities can be dealt with immediately. The accompanying measures are implemented 100% during the next iterations. EGOV: This does not fit with some project methodologies used nowadays. The mind shift is not easy to implement. Fortunately successful planning games are easy to produce and help in convincing the other stakeholders.
Small Releases: Many releases are produced due to the short iterations of 3-5 weeks. The focus is on 100% of the agreed functionality.	CO: This upsets usually the whole roll-out process. SE: The high quality of the implemented features helps with the security. IM: The security levels increase fast.
Customer Tests: The XP customer defines one or more automated acceptance tests for each feature.	CO: A good test script is essential to assure the quality. SE: The improved quality and specifically defined security tests will help a great deal in ensuring the overall security goals. IM: Automated security tests (a test suite) can be implemented that way, e.g. the common criteria [29] or the functions of a different standard could be integrated in the same way.
Simple Design: XP evolves software from simple designs to exactly the necessary complexity. Designing is integrated into the whole cycle.	CO: This is contradictory to the traditional analysis first approach. XP assumes that the future cannot be really predicted. SE: The simpler a system the less errors it can contain. IM: Implementing the simple principles first and evolving the complexity later will help the stakeholders to become familiar with the security. EGOV: This principle is a paradigm shift for the normal process of public administrations.
Pair Programming: At all times two programmers develop the code together.	CO: This is one of the hardest principles to implement due to the resistance of about half the programmers. SE: The built-in quality check pair programming provides can be used also to check for better security. IM: The approach works also in the development of security. By means of bilateral questioning, each other's possible gaps can be

Core practice and rules	Security adapted version
	identified. It is already known as the four-eye principle.
Test-Driven Development: First the tests are written and then the software. Unit tests run almost all the time and integration happens at least once a day.	CO: An unconventional but powerful concept. SE: This goes well with standards like the Common Criteria. IM: This goes well with standards like the Common Criteria. EGOV: For high-security applications the test suite is developed by a different company or branch.
Design Improvement: With Refactoring [30]. One of the goals of Extreme Programming is to keep the costs of re-factoring at a linear level.	CO: Refactoring is a problem in the software world. SE: Refactoring could create a security problem, if the tests are not designed in a way to detect problems. IM: When the security processes and workflows are dealt with that way, it is assured that the awareness does not fade. EGOV: Due to the many stakeholders the requirements change more rapidly in eGovernment projects than in projects related to eBusiness. An easier refactoring therefore comes in handy.
Continuous Integration: The system should be integrated at all times.	CO: This needs a new development environment and an attitude of the programmers to follow it. SE: There is no specific value for security in that feature. IM: This cannot be really applied to the security implementation.
Collective Code Ownership: Every programmer can change code everywhere.	CO: Without pair programming this will remain merely an illusion. The unwillingness to touch other programmer's code will, however, be diminished. SE: The code gets simpler and better documented. This means that problematic areas are eliminated. IM: A deeper awareness of the overall scheme can be reached. EGOV: The knowledge is more distributed reducing the dependence on individual programmers.
Coding standard: There is a coding standard and it is adhered to very strictly.	CO: This is an old practice that works best. The enforcing is mainly a problem, especially when software engineering is not a core process of the company. SE: Also here quality and with it security improves. IM: This will mainly relate to documentations. It means that the documents will be understood. EGOV: Most branches of the administration are not really software factories. Therefore more elaborate work on good coding is certainly useful.
Metaphor: Extreme programming teams develop a common vision of how the program works. These are simple evocative descriptions.	CO: This is a simplification of the architecture part of other design methodologies. Some parts should remain in an architecture step/sub process. SE: Here quality and with it security improves as well. IM: Security must be sold to the users. Metaphors are a great means in achieving this. EGOV: The really needed common vision in security can be established that way more easily.
Sustainable Pace: The teams work hard, but at a pace that can be sustained indefinitely. This means overtime is avoided whenever possible.	CO: A good management practice. SE: Not really security relevant. IM: Not really security relevant.
User stories are written.	CO: These stories can replace "use cases" in other methodologies. SE: The stories can have a negative security impact if the customer does not understand the security issues at hand and focuses on other aspects. IM: This is what security functions are about. Therefore the story principle can naturally be integrated in security design.
Project velocity is measured.	CO: It is important to measure important things and not to "torture"

Core practice and rules	Security adapted version
	the staff (which is the standard perception when one is not very careful about it.). The purpose is to have feedback and to learn from it. SE: A better control over the process improves quality and functionality and therefore security. IM: Security always has to balance risk with cost. Measuring in shorter cycles may really help to improve the security per dollar.
Move people around.	CO: People must believe in their project. Moving them around helps a great deal to achieve this. SE: No special impact. IM: Improves the effectiveness of the security team.
A Stand-up meeting starts each day.	CO: This is also a tried and tested practice that works out well. SE: Improves the focus of the team. IM: Improves the focus of the team.
Fix XP when it breaks.	CO: Kaize built directly into the process. SE: No special effect. IM: No special effect.
Use CRC cards [31] for design sessions.	CO: Ad hoc documentation. SE: No special effect. IM: Results in fast documentation.
Create spike solutions to reduce risk.	CO: Prototyping is known to be important to reduce the risk at the beginning. SE: Risk reduction. IM: A way to tackle the heavy security issues. EGOV: eGovernment is new. Therefore a lot of risks are involved. Spike solutions help to reduce the risks.
No functionality is added at an early stage.	CO: Solve the problems when they occur. Because nobody can imagine the future needs and problems, functionality to cope with these unknown needs will almost certainly not be necessary as the requirements change day by day. SE: This principle decreases the total project size and therefore increases security. IM: KISS is certainly also an issue to improve security in general. EGOV: Public administrations tend to go for general solutions. This principle is in conflict with the working method.
Leave optimisation until last.	CO: This should not prevent thinking about scaling up to a productive level at the end. SE: Improves quality over design size. IM: Cannot be applied.
When a bug is found tests are created.	CO: Prevents a bug from survive and happen again in the next rebuild. SE: Enhances quality. Fixes stay fixed. IM: This creates a sort of emergency response team.
The score of acceptance tests is published.	CO: Creates awareness and motivates the team. SE: Improves quality. IM: Creates awareness.

Table 1: XP core practices and rules and relevancy/adaption for security

10. Drawbacks

Barry Boehm [32] made a comparison between traditional project management and extreme programming, showing when to use which methodology. In general the conclusions of the article can be applied to security as well. However, usually the team responsible

for creating the security is rather small and therefore more on the XP side. Another drawback is that XP does not focus on the interaction of the customer with his/her co-workers in detail. In most cases therefore, "less extreme programming" will be the path to follow.

11. Conclusions

There are lessons to be learnt from iterative development that can be used in security development. Security engineers like to base their work on mathematical foundations. It should be mentioned that not even the best cryptographic algorithms used today are proven mathematically safe (In quantum computation this will become true [33]). The IT systems used in public administrations are complex [34] and for this reason a complete analysis in advance without implementation and evolution is not possible. The adherence to standards is a good idea, but it should be used not as first aid, but as a long term project. The process as shown in figure 4 is adopted more frequently in order to obtain early results.

The main goals of this process must be:

- Build security into the core and hearts in the end
- Improve the software development cycle
- Change procurement to include security
- Change project management priorities to include security (and quality)

Security becomes more and more important, a fast reliable path to security is worth adding some new principles.

12. References

- [1] R. Oppliger: Computersicherheit - eine Einführung, Vieweg 1992, page 13
- [2] M. Gisler: Einführung in die Begriffswelt des eGovernment, M. Gisler, D. Spahni. Egovernment: eine Standortbestimmung, Haupt, 2. Edition, 2001, page 16
- [3] <http://www.commoncriteria.org/faq/definitions.html>
- [4] M. Günter, Sicherheit im eGovernment, to be published in the proceedings of the 2. eGovernment-Symposiums 2001
- [5] Haberfellner et al, Systems Engineering, Methodik und Praxis, Verlag Industrielle Organisation, 1999, page 5
- [6] <http://www.isb.admin.ch/architektur/standards/inventar/deutsch/p010/aktuell/p010-s02.html>
- [7] <http://www.bsi.de/gshb/>
- [8] M. Günter, Sicherheit im eGovernment, to be published in the proceedings of the 2. eGovernment-Symposiums 2001
- [9] <http://sol.brunel.ac.uk/~jarvis/bola/culture/harrison.html>
- [10] <http://litc.sbu.ac.uk/jcalt/conference.html>
- [11] <http://www.sans.org/mistakes.htm>
- [12] <http://www.microsoft.com/windows2000/server/evaluation/news/bulletins/win2kcomcrit.asp>
- [13] <http://www.stosdarwin.org/projects/se-darwin/se-darwin.html>
- [14] <http://security.dstc.edu.au/projects/corba/>
- [15] <http://www.alphaworks.ibm.com/tech/xmlsecuritysuite>
- [16] http://www.epic.org/crypto/export_controls/itar.html
- [17] A. Sharp, P. McDermott: Workflow Modelling: Tools for Process Improvement and Application Development. Artech House, 2001, page 58
- [18] S. Jablonski: Workflow-Management-Systeme: Modellierung und Architektur, Thomson Publ., 1995
- [19] <http://www.cert.org/archive/pdf/lifecycle-models.pdf>
- [20] G. Bruce, R. Dempsey: Security in Distributed Computing, Hewlett-Packard Professional Books, 1996, page 48
- [21] e.g. <http://www.sysedco.com/library/tech/0004.htm>
- [22] <http://www.computer.org/internet/v5n6/index.htm>
- [23] http://interactive.sei.cmu.edu/news@sei/columns/the_architect/2001/1q01/architect-1q01.pdf
- [24] http://asd-www.larc.nasa.gov/barkstrom/public/The_Standard_Waterfall_Model_For_Systems_Development.htm
- [25] T. A. Pardo, J. H. Scholl, "Walking Atop the Cliffs: Avoiding Failure and Reducing Risk in Large Scale E-Government Projects", presented at HICSS 35, Hawaii, 2001.
- [26] <http://www.extremeprogramming.org/rules/iterative.html>
- [27] K. Beck: Extreme Programmierung, Addison-Wesley, 2000, page xiii
- [28] e.g. K. Beck, M. Fowler: Extreme Programming planen. Addison-Wesley, 2001
- [29] Common Criteria for Information Technology Security Evaluation, Version 1.0, Bundesamt für Sicherheit in der Informationstechnik, 30. 1. 1996
- [30] M. Fowler, "Refactoring: Improving the Design of Existing Code", Addison-Wesley, 1999
- [31] <http://members.aol.com/humansandt/techniques/crc.htm>
- [32] B. Boehm: Get Ready for Agile Methods with Care, Computer, 1/2002, page 64ff
- [33] <http://www.aip.org/enews/physnews/2000/split/pnu480-1.htm>
- [34] A. Ninck et al: Systemik - Integrales Denken, Konzipieren und Realisieren, Verlag Industrielle Betriebe, 2nd edition, 1998, page 48ff

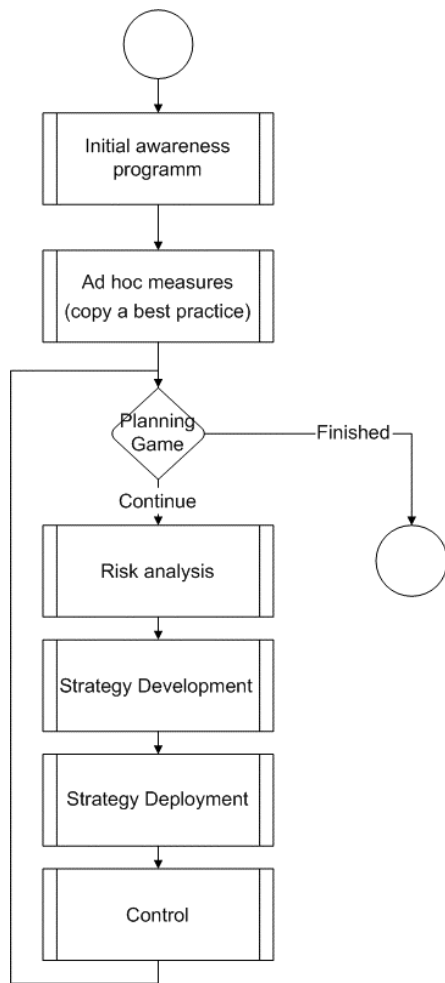


Figure 4: Suggested path for implementing security in a complex environment